

ENGENHARIA DE SOFTWARE · FCTE / UnB

Engenharia de Dados

Fontes, formatos, modelos, armazenamento e movimentação de dados

Banco de Dados 2

Módulo: ciclo de vida do dado

Versão didática em português

Adaptado e ampliado a partir de "Data Engineering 101" — Chip Huyen, CS 329S (Stanford)

Esta aula é um panorama — e isso é proposital

Cada um dos quatro blocos abaixo poderia ser uma disciplina inteira. Aqui você ganha o mapa; a profundidade vem nos módulos seguintes e na bibliografia.

1

O que você VAI levar

Vocabulário, os trade-offs centrais e a capacidade de justificar escolhas.

2

O que NÃO cabe aqui

Ajuste fino de índices, tuning de consultas, administração e operação de SGBDs.

3

Como estudar

Cada bloco fecha com uma pergunta de checagem e um exercício curto.



Regra do curso: toda escolha de tecnologia precisa de justificativa registrada. O que você aprende aqui alimenta diretamente os ADRs que vamos escrever.

O ciclo de vida do dado

Todo dado percorre o mesmo caminho. Hoje olhamos as quatro primeiras etapas — o que acontece antes de alguém "só fazer um SELECT".



Cada seta é uma decisão de projeto: o que coletar, em que formato guardar, com que modelo representar e quando transformar.

Errar cedo no ciclo custa caro depois — dado mal coletado não se conserta com consulta esperta.

Quatro perguntas, quatro blocos

Bloco	Pergunta que o bloco responde	Conceitos centrais
1. Fontes de dados	De onde vem o dado e o que isso implica?	Dados de usuário, de sistema, internos e de terceiros; rastreamento
2. Formatos de dados	Como gravar o dado para poder lê-lo depois?	Serialização; JSON, CSV, Parquet, Avro, Protobuf, Pickle; linha vs. coluna; texto vs. binário
3. Modelos de dados	Como representar o mundo dentro do banco?	Relacional, normalização, SQL declarativo; NoSQL: documento e grafo; estruturado vs. não estruturado
4. Engines e processamento	Que tipo de carga o banco precisa aguentar?	OLTP, ACID, OLAP; desacoplamento de armazenamento e processamento
5. Movimentação	Como o dado vai da origem ao consumo?	ETL, ELT e o pêndulo entre os dois



Fio condutor: não existe "melhor banco". Existe a escolha que respeita a fonte, o formato, o modelo e a carga de trabalho do seu problema.

BLOCO 1

Fontes de dados

- - Quem (ou o quê) gera o dado
 - Dados de usuário vs. dados de sistema
 - Bancos internos da organização
 - Dados de terceiros e o fim dos IDs de rastreamento

De onde vêm os dados?

A origem do dado determina quase tudo o que vem depois: volume, qualidade, urgência de processamento e restrições legais.

1

Gerados por usuários

Tudo que uma pessoa insere deliberadamente: formulários, textos, uploads, avaliações, cliques em "enviar".

2

Gerados por sistemas

Logs, eventos, metadados, previsões de modelos, telemetria de infraestrutura. Ninguém digitou — a máquina emitiu.

3

Bancos internos

O que a organização já mantém: usuários, estoque, contratos, relacionamento com clientes, cadastros.

4

Dados de terceiros

Comprados ou licenciados de fora: dados de mercado, demográficos, comportamentais agregados.

?

Pergunta prática: para cada fonte, quem é o dono do dado, com que frequência ele chega e o que acontece se ele vier errado?

Cada fonte é uma torneira diferente

Antes do termo técnico: pense em cada fonte de dados como uma torneira ligada ao seu sistema. Três coisas importam.

1

Vazão

Quanto dado chega por segundo? Um formulário de matrícula pinga; um log de servidor jorra.

Impacto: dimensionamento de armazenamento e de rede.

2

Pureza

O que sai da torneira já vem limpo? Texto digitado por gente vem sujo; log emitido por código vem padronizado.

Impacto: quanto esforço de validação e limpeza.

3

Previsibilidade

O formato muda sem aviso? Uma API de terceiro pode mudar amanhã; seu próprio log você controla.

Impacto: fragilidade do pipeline e contratos de dados.

!

Se você não sabe responder essas três perguntas sobre uma fonte, você ainda não pode desenhar o pipeline que a consome.

Dados de usuário vs. dados de sistema

Critério	Gerados por usuários	Gerados por sistemas
O que são	Entradas deliberadas: texto, cliques em botões, uploads, formulários	Logs, eventos, metadados, previsões de modelos
Qualidade do formato	Facilmente mal formatados (campo vazio, data errada, emoji no CPF)	Mais fáceis de padronizar — o formato é definido no código
Urgência	Precisam ser processados o quanto antes: o usuário está esperando resposta	Podem ser processados periodicamente (em lote)
Exceção da urgência	—	Quando servem para detectar problemas: aí têm de ser vistos rápido
Volume	Cresce com o número de pessoas usando	Cresce muito rápido e sozinho — cada requisição vira várias linhas
Descarte	Normalmente preservados (é o conteúdo do produto)	Podem ser apagados quando deixam de ser úteis



Consequência de projeto: dado de usuário puxa requisitos de latência; dado de sistema puxa requisitos de volume e de política de retenção.

Duas nuances que confundem todo mundo

Dado comportamental é dos dois mundos

Cliques, rolagem de tela, tempo em cada página, o que a pessoa assistiu até o fim.

Quem emite é o sistema (a pessoa não "digitou" nada), mas o dado é sobre a pessoa — e por isso é tratado como dado de usuário para efeito de privacidade e de consentimento.

Log cresce mais rápido do que você imagina

Uma única requisição pode gerar dezenas de linhas de log em serviços diferentes.

Existe um ecossistema inteiro só para coletar, indexar e analisar log: Logstash / Elastic, DataDog, Logz.io, Grafana Loki, OpenSearch.

Decisão sobre logs	Pergunta a fazer	Resposta típica
Guardar tudo?	Esse log responde alguma pergunta real?	Não. Defina o que é útil antes de ligar a torneira.
Por quanto tempo?	Qual o prazo em que ele ainda tem valor?	Política de retenção por tipo (7 dias, 90 dias, 1 ano).
Processar quando?	Alguém precisa disso agora?	Em lote — exceto detecção de falha e de fraude.

Dados de terceiros: fascinante e desconfortável

Empresas inteiras (data brokers) existem para coletar dados sobre pessoas que não são suas clientes e revendê-los agregados.

1

Tipos de dado

Atividade em redes sociais, faixa de renda, profissão, hábitos de compra, deslocamento.

2

Grupo demográfico

O produto raramente é "a pessoa X". É um segmento: "homens, 25-34 anos, que trabalham com tecnologia".

3

Identificador móvel

O MAID (Mobile Advertiser ID) permitia costurar comportamento entre aplicativos diferentes do mesmo aparelho.

4

Por que interessa

Vira atributo (feature) de modelo: "quem gosta de A também gosta de B" — recomendação e segmentação.

§

No Brasil isso não é neutro: dado pessoal de terceiro exige base legal sob a LGPD. "Comprei de um fornecedor" não é base legal.

O fim dos IDs de rastreamento... ou não?

1. O modelo antigo

Cada aparelho carregava um identificador estável (IDFA, cookie de terceiros).

Qualquer aplicativo podia lê-lo e cruzar o comportamento entre serviços.

Dado de terceiro abundante e barato



2. A virada regulatória

Consentimento explícito por aplicativo (App Tracking Transparency), fim dos cookies de terceiros, GDPR e LGPD.

A maioria das pessoas diz "não".

A torneira fecha



3. ...ou será?

Surgem contornos: identificadores próprios, impressão digital do navegador, login obrigatório, dados de primeira mão.

O incentivo econômico não mudou



Lição de engenharia: um pipeline que depende de uma fonte externa depende também da regulação e do modelo de negócio dela. Trate isso como risco de arquitetura, não como detalhe jurídico.

Classifique cada dado abaixo

a

a) Latência de resposta da API

Registrada automaticamente a cada requisição pelo servidor.

b

b) Comentário em um chamado

Texto livre escrito por uma pessoa no formulário de suporte.

c

c) Tempo assistindo a um vídeo

Medido pelo player enquanto a pessoa assiste.

d

d) Faixa de renda estimada

Comprada de um fornecedor de dados de mercado.

?

Dica: pergunte "quem emitiu?" e depois "sobre quem é?". As duas respostas nem sempre coincidem — e é exatamente aí que mora a diferença entre dado de sistema e dado comportamental.

Inventário de fontes do seu projeto

Passo	O que fazer	Entregável
1	Liste 5 fontes de dados desse sistema	Uma linha por fonte
2	Classifique cada uma: usuário, sistema, interna ou terceiro	Rótulo na linha
3	Para cada uma, estime vazão, pureza e previsibilidade (alta / média / baixa)	Três colunas preenchidas
4	Marque quais contêm dado pessoal e qual seria a base legal	Coluna "LGPD"
5	Aponte a fonte mais frágil e diga o que quebra se ela falhar	Um parágrafo curto



Esse inventário é a primeira seção do ADR de escolha de banco que vocês vão escrever no módulo seguinte.

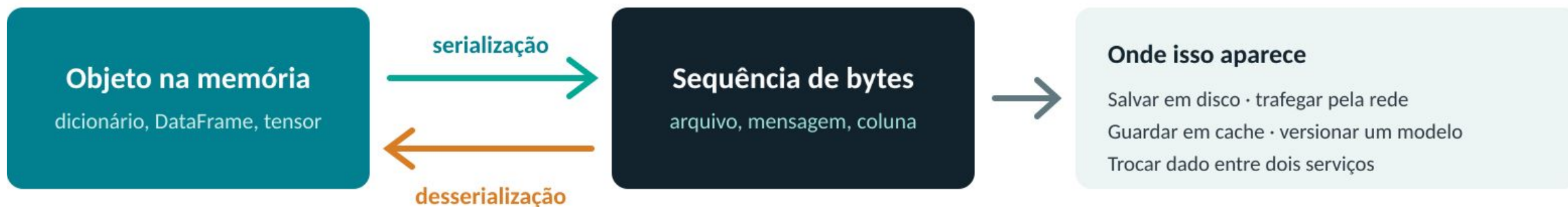
BLOCO 2

Formatos de dados

- - Serialização e desserialização
 - JSON, CSV, Parquet, Avro, Protobuf, Pickle
 - Orientação a linha vs. orientação a coluna
 - Formatos de texto vs. formatos binários

Guardar dado só interessa se você for lê-lo depois

Parece óbvio, mas é a definição inteira do problema: o formato existe para que o que foi gravado possa ser reconstruído do outro lado.



Definição para anotar: formato de dados é um padrão acordado de serialização, para que o dado possa ser transmitido e reconstruído depois. "Acordado" é a palavra-chave — quem escreve e quem lê precisam concordar.

Formato é embalagem para mudança

Você tem um armário montado (o objeto na memória) e precisa mandá-lo para outra cidade (disco, rede, outro serviço).

1

Desmontar e etiquetar tudo à mão

Cada peça embrulhada, com etiqueta escrita em português.

Qualquer pessoa entende sem manual. Ocupa muito espaço no caminhão.

É o formato de texto: CSV, JSON.

2

Embalar a vácuo com código de peça

Compacto: cada peça vira um código.

Sem o manual (o esquema) ninguém remonta nada. Cabe muito mais no caminhão.

É o formato binário: Parquet, Avro, Protobuf.

3

Guardar por cômodo ou por tipo de peça

Caixa "quarto inteiro" vs. caixa "todos os parafusos da casa".

Uma é boa para montar um cômodo; a outra, para contar parafusos.

É linha vs. coluna — o próximo tema.

?

Nenhuma das três é "melhor". A pergunta certa é: quem vai abrir a caixa, com que frequência e procurando o quê?

Três perguntas antes de escolher um formato

1. Como guardar dado multimodal?

Um mesmo registro pode misturar matriz de pixels, rótulo textual e identificador numérico. O formato precisa acomodar tudo isso junto — ou você acaba com três arquivos que precisam ser mantidos em sincronia.

2. Qual o padrão de acesso?

Com que frequência o dado será lido? Lê-se o registro inteiro ou só duas colunas? Lê-se uma vez por dia ou mil vezes por segundo? Escrever muito e ler pouco pede um formato; o contrário pede outro.

3. Em que hardware isso vai rodar?

Modelos complexos rodando em CPU, GPU ou TPU têm exigências diferentes de leitura. Formato ruim vira gargalo de E/S: a placa cara fica ociosa esperando o disco.

```
Exemplo de registro multimodal:  
{ 'image': [[200,155,0], [255,255,255], ...], 'label': 'carro', 'id': 1 }
```

Os formatos que você vai encontrar na prática

Formato	Binário ou texto	Legível por humano	Onde aparece	Ponto forte
JSON	Texto	Sim	Em toda parte: APIs, configuração, logs	Flexível, aninhado, universal
CSV	Texto	Sim	Em toda parte: planilhas, exportações, dados abertos	Simplicidade absoluta
Parquet	Binário	Não	Hadoop, Spark, Amazon Redshift, data lakes	Orientado a coluna, compressão alta
Avro	Binário (primário)	Não	Hadoop, Kafka	Esquema embutido, evolui bem
Protobuf	Binário (primário)	Não	Google, TensorFlow (TFRecord), gRPC	Compacto e muito rápido
Pickle	Binário	Não	Python, serialização do PyTorch	Serializa qualquer objeto Python



Cuidado com Pickle: ele executa código ao ser desserializado. Nunca abra um arquivo .pkl de origem desconhecida — é uma porta de entrada clássica para execução remota de código.

Orientação a linha vs. orientação a coluna

A tabela é um retângulo na sua cabeça, mas o disco é uma fita: os bytes saem em fila. A pergunta é qual ordem escolher.

	Nome	Cidade	Preço
reg 1	Ana	Brasília	20
reg 2	Beto	Recife	30
reg 3	Caio	Belém	15

A mesma tabela lógica...

Gravada por LINHA

Ana | Brasília | 20 | Beto | Recife | 30 |
Caio | Belém | 15

Boa para pegar um registro

"me traga tudo sobre o pedido 4271"
Uma leitura contígua resolve.

Gravada por COLUNA

Ana | Beto | Caio | Brasília | Recife |
Belém | 20 | 30 | 15

Boa para pegar um atributo

"qual o preço médio de tudo?"
Lê só a coluna preço, ignora o resto.



Regra de bolso: linha favorece quem trabalha com amostras (registros inteiros); coluna favorece quem trabalha com atributos (uma variável de todos os registros).

Por que isso muda o desempenho de verdade

Aspecto	Orientado a linha	Orientado a coluna
Leitura típica	Registro completo, poucos por vez	Poucas colunas, milhões de registros
Adicionar um registro novo	Barato: acrescenta no fim	Caro: mexe em todas as colunas
Adicionar uma coluna nova	Caro: reescreve todos os registros	Barato: acrescenta um bloco
Compressão	Pior: tipos misturados lado a lado	Muito melhor: valores do mesmo tipo e repetidos ficam juntos
Ler 3 de 200 colunas	Lê as 200 e descarta 197	Lê só as 3 — economia enorme de E/S
Exemplos	CSV, Avro, bancos transacionais	Parquet, ORC, warehouses analíticos



Aqui nasce a ligação com o bloco 4: sistemas transacionais tendem a ser orientados a linha e sistemas analíticos, a coluna. Não é coincidência — é consequência do padrão de acesso.

A armadilha do dia a dia: DataFrame vs. ndarray

NumPy ndarray

Orientado a LINHA por padrão.

É possível pedir explicitamente a ordem por coluna (order="F").

Acessar uma linha inteira é rápido e contíguo.

pandas DataFrame

Orientado a COLUNA.

Acessar uma linha é bem mais lento do que acessar uma coluna — e mais lento do que a operação equivalente em NumPy.

Por isso iterar linha a linha em DataFrame é tão lento.

```
Lento:    for i, linha in df.iterrows(): ...           # atravessa a orientação do DataFrame
Rápido:   df["preco"].mean()                         # trabalha na direção natural, a coluna
```



Não é "pandas é lento". É que você pediu uma amostra a uma estrutura organizada por atributo. Conhecer o formato explica o desempenho.

Texto vs. binário: o preço da legibilidade

	Arquivos de texto	Arquivos binários
Exemplos	CSV, JSON, XML, YAML	Parquet, Avro, Protobuf, Pickle
Vantagem	Legível por humanos — abre em qualquer editor, é fácil de depurar e de versionar	Compacto — ocupa menos disco, menos rede e menos tempo de leitura
Guardar o número 1.000.000	7 caracteres → 7 bytes (mais separadores e aspas)	Como int32, 4 bytes — quase metade
Custo escondido	Cada número precisa ser convertido de texto na leitura	Precisa do esquema ou da biblioteca certa para ser lido
Quando escolher	Troca com humanos, dados abertos, configuração, volume pequeno	Volume grande, leitura repetida, pipeline entre máquinas



Multiplique por bilhões de linhas: 3 bytes de diferença por número viram terabytes de disco e horas de leitura. Legibilidade é confortável — e cara em escala.

Qual formato você escolheria?

a

a) CSV

Texto, orientado a linha,
legível, universal.

b

b) JSON

Texto, aninhado, flexível,
muito verboso.

c

c) Parquet

Binário, orientado a coluna,
compressão alta.

d

d) Pickle

Binário, específico de Python,
serializa objetos.

?

Cenário: 4 bilhões de registros de sensores, 60 colunas. As análises quase sempre calculam médias de 2 ou 3 colunas por vez, e ninguém precisa abrir o arquivo no editor.

Medir, não adivinhar

Roteiro de laboratório curto. Use qualquer base pública com pelo menos 1 milhão de linhas.

Passo	O que fazer	O que observar
1	Salve a mesma base em CSV e em Parquet	Tamanho em disco dos dois arquivos
2	Cronometre a leitura completa de cada um	Tempo total de carga
3	Cronometre a leitura de apenas 2 colunas em cada um	A diferença aqui é a maior de todas
4	Calcule a média de uma coluna nos dois casos	Tempo de consulta analítica
5	Escreva 3 linhas explicando os números com o que vimos	Sua justificativa vira um ADR



Entregável: uma tabela com os quatro tempos e um parágrafo de conclusão. Números medidos valem mais do que opinião sobre desempenho.

BLOCO 3

Modelos de dados

- - O modelo relacional e a normalização
 - SQL: linguagem declarativa
 - NoSQL: modelo de documento e modelo de grafo
 - Estruturado vs. não estruturado; warehouse vs. lake

Modelo de dados é a lente, não o território

Um modelo de dados descreve como o dado é representado. Ele não muda a realidade — muda quais perguntas sobre ela ficam fáceis ou difíceis.

1

Mapa do metrô

Ignora distâncias reais e desenha linhas retas.

Ótimo para: "como vou da estação A à B?"
Péssimo para: "quanto vou caminhar?"

2

Mapa de relevo

Mostra altitude e terreno, ignora ônibus e ruas.

Ótimo para: "onde alaga?"
Péssimo para: "por onde eu passo de carro?"

3

Modelo de dados

Mesmo dado, recortes diferentes.

Escolher um modelo é escolher quais consultas serão baratas — e aceitar que outras ficarão caras.



Dois paradigmas principais organizam quase tudo o que existe: o modelo relacional (1970) e o guarda-chuva NoSQL. Vamos aos dois.

O modelo relacional (1970) e ainda de pé

Relação é uma tabela: um conjunto de tuplas com o mesmo cabeçalho.

Tupla (linha) é um registro — e o conjunto NÃO é ordenado: não existe "a terceira linha".

Atributo (coluna) também não é ordenado: você o identifica pelo nome, não pela posição.

Cabeçalho define nome e tipo de cada atributo — é o esquema da relação.

Formatos usuais CSV e Parquet armazenam bem dados nesse modelo.

Anatomia de uma relação

Atributo 1	Atributo 2	Atributo 3	Atributo 4
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

Tuplas: conjunto não ordenado

Atributos: também não ordenados



Consequência prática: se o seu código depende da ordem das colunas de um SELECT *, ele já está errado do ponto de vista do modelo.

Normalização: o problema

Relação original de livros. Repare no que se repete: a editora e o país aparecem em várias linhas.

Título	Autor	Formato	Editora	País	Preço
Harry Potter	J.K. Rowling	Brochura	Banana Press	Reino Unido	R\$ 20
Harry Potter	J.K. Rowling	E-book	Banana Press	Reino Unido	R\$ 10
Sherlock Holmes	Conan Doyle	Brochura	Guava Press	EUA	R\$ 30
O Hobbit	J.R.R. Tolkien	Brochura	Banana Press	EUA	R\$ 30
Sherlock Holmes	Conan Doyle	Brochura	Guava Press	EUA	R\$ 15



E se a "Banana Press" mudar de nome para "Pineapple Press"? Você precisa alterar 3 linhas. Se esquecer uma, o banco passa a afirmar duas verdades diferentes sobre a mesma editora.

Isso se chama anomalia de atualização — e é o que a normalização evita.

Normalização: a solução

Separe o que fala de livro do que fala de editora, e ligue as duas por uma chave.

Relação Livro (atualizada)

Título	Autor	Formato	ID Editora	Preço
Harry Potter	J.K. Rowling	Brochura	1	R\$ 20
Harry Potter	J.K. Rowling	E-book	1	R\$ 10
Sherlock Holmes	Conan Doyle	Brochura	2	R\$ 30
O Hobbit	J.R.R. Tolkien	Brochura	1	R\$ 30
Sherlock Holmes	Conan Doyle	Brochura	2	R\$ 15

Relação Editora

ID Editora	Editora	País
1	Banana Press	Reino Unido
2	Guava Press	EUA



Agora "Banana Press" existe em um lugar só. Trocar o nome é alterar uma célula — e todos os livros passam a refletir a mudança automaticamente.

Normalização: o que você ganha e o que você paga

Vantagens

- Menos erros: a grafia fica padronizada em um único lugar.
- Mais fácil de atualizar: um campo, uma alteração.
- Mais fácil de traduzir e localizar: mude "Reino Unido" uma vez e vale para tudo.
- Menos espaço desperdiçado com repetição.

Desvantagem

- Junções (JOIN) entre várias tabelas grandes são lentas.

O dado que "andava junto" agora precisa ser remontado a cada consulta. Em escala analítica, isso pesa.

Por isso warehouses frequentemente desnormalizam de propósito: aceitam repetição em troca de leitura rápida.



Não existe "normalizar é certo". Existe: sistema transacional tende a normalizar (escrita correta); sistema analítico tende a desnormalizar (leitura rápida).

Modelo relacional $\neq$ modelo SQL

São coisas diferentes.

O modelo SQL difere um pouco do modelo relacional teórico. O exemplo clássico: uma tabela SQL pode conter linhas duplicadas — uma relação verdadeira, por ser um conjunto, não pode.

SQL é uma linguagem de consulta.

É o jeito de especificar quais dados você quer de um banco. Não é o modelo em si; é como você conversa com ele.

SQL é declarativa.

Você diz ao sistema O QUE quer, não COMO obter. Compare com uma linguagem imperativa, em que você escreveria o laço de repetição você mesmo.

Quem decide o "como" é o otimizador.

O sistema escolhe a ordem das junções, quais índices usar, se paraleliza. Isso se chama otimização de consultas — e é uma das razões de o SQL ter sobrevivido 50 anos.



Efeito colateral bom: como você não descreveu o "como", o banco pode ficar mais rápido a cada nova versão sem que você reescreva uma linha da sua consulta.

SQL

Aprenda SQL. De verdade.

É a ferramenta essencial de quem trabalha com dados — e a de maior retorno por hora de estudo em toda a área.

Além do SELECT básico

Junções, agregações, GROUP BY, HAVING

Funções de janela

Ranking, médias móveis, comparação com a linha anterior

CTEs e subconsultas

Consultas legíveis e depuráveis

Plano de execução

Ler o EXPLAIN e entender por que está lento

Onde o esquema rígido começa a doer

O esquema é uma promessa: "toda linha desta tabela tem exatamente estes campos, com estes tipos". A promessa protege — e aprisiona.

E se eu precisar de uma coluna nova?

É preciso alterar a tabela (ALTER TABLE). Em uma tabela com bilhões de linhas isso pode travar escritas, exigir janela de manutenção e quebrar todo código que fazia SELECT *.

E se o tipo de uma coluna mudar?

Um campo que era inteiro agora precisa aceitar texto. É migração de dados: converter tudo o que já existe, sem perder nada e sem derrubar a aplicação que está lendo.



Quando o formato do dado muda com frequência — e a organização quer velocidade acima de garantia — a rigidez vira o gargalo. É desse desconforto que nasce o movimento NoSQL.

Atenção: "esquema rígido incomoda" não significa "esquema é ruim". Na maioria dos sistemas corporativos, a rigidez é justamente o que impede o banco de virar lixo.

NoSQL: de "No SQL" para "Not Only SQL"

O nome pegou como oposição, mas a leitura que ficou é outra: não apenas SQL. São modelos alternativos para casos em que o relacional atrapalha.

1

Modelo de documento

Conceito central: o documento — um registro autocontido (tipicamente JSON) que carrega tudo o que descreve uma entidade.

Relacionamentos entre documentos são raros e desencorajados.

Exemplos: MongoDB, Couchbase, Firestore, DocumentDB.

2

Modelo de grafo

Conceito central: o grafo — nós (entidades) e arestas (relações entre elas).

Aqui o relacionamento é o que importa mais: ele é cidadão de primeira classe, com tipo e propriedades.

Exemplos: Neo4j, Neptune, JanusGraph.



Diferença em uma frase: no modelo de documento a pergunta é "o que eu sei sobre esta entidade?"; no modelo de grafo é "como estas entidades estão conectadas?".

Modelo de documento: os mesmos livros

Cada livro vira um documento autocontido. Repare: a editora aparece dentro do documento, repetida — desnormalização deliberada.

```
{ "titulo": "Harry Potter",  
  "autor": "J.K. Rowling",  
  "formato": "brochura",  
  "preco": 20,  
  "editora": {  
    "nome": "Banana Press",  
    "pais": "Reino Unido" } }
```

```
{ "titulo": "Sherlock Holmes",  
  "autor": "Conan Doyle",  
  "formato": "brochura",  
  "preco": 30,  
  "editora": {  
    "nome": "Guava Press",  
    "pais": "EUA" } }
```

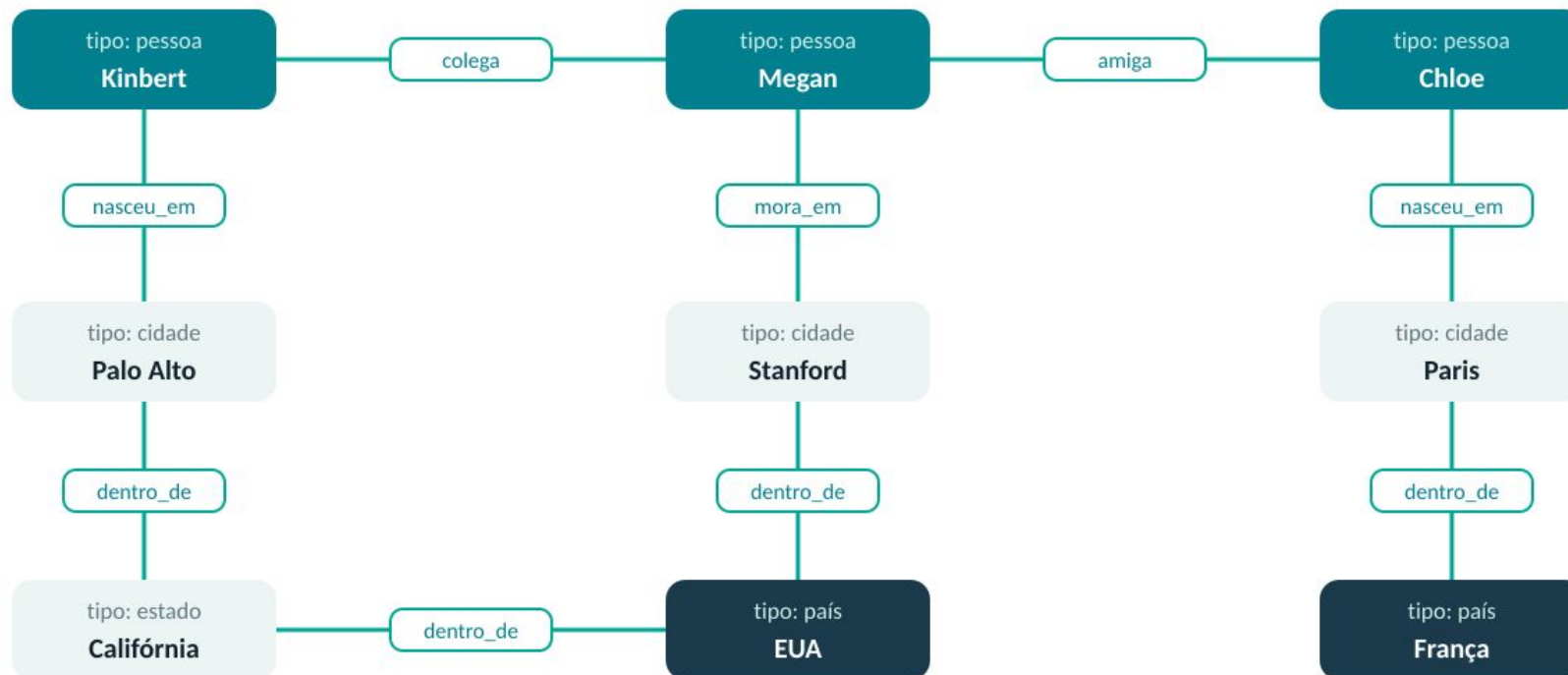
Vantagens

- Leitura em uma tacada: tudo sobre o livro em um só lugar, sem junção.
- Cada documento pode ter campos diferentes — esquema flexível.
- Fácil de mapear para objetos do código.

Custos

- Voltou a anomalia de atualização: renomear a editora exige varrer documentos.
- Consultas que cruzam documentos são fracas ou inexistentes.
- A responsabilidade pela consistência migra para a aplicação.

Modelo de grafo: relações em primeiro plano



Tudo é tipado: os nós têm tipo (pessoa, cidade, estado, país) e as arestas também (nasceu_em, mora_em, dentro_de, colega). A relação carrega significado.

A consulta que revela a diferença

"Mostre todas as pessoas que nasceram nos Estados Unidos."

No grafo: fácil

Parta do nó "EUA" e caminhe as arestas "dentro_de" no sentido inverso, até chegar às cidades; depois siga as arestas "nasceu_em".

A profundidade da hierarquia não importa: cidade → estado → país ou cidade → país, o mesmo percurso funciona.

No SQL: difícil

Cidade fica em estado, que fica em país — mas nem sempre: às vezes cidade fica direto no país, às vezes há região no meio.

Como o número de saltos varia, você precisa de junções recursivas (WITH RECURSIVE) ou de uma tabela para cada nível.



Moral do bloco 3: o modelo certo é aquele em que a sua consulta mais frequente é a mais simples de escrever. Se a sua pergunta central é sobre conexões, force o relacional e você vai sofrer.

O mapa completo dos modelos (complemento de BD2)

Modelo	Unidade básica	A pergunta em que ele brilha	Exemplos
Relacional	Tupla em relação	Consultas variadas com integridade e transações	PostgreSQL, MySQL, Oracle
Documento	Documento JSON	"Tudo sobre esta entidade", esquema variável	MongoDB, Couchbase, Firestore
Chave-valor	Par chave → valor	Busca por chave em latência mínima; cache e sessão	Redis, DynamoDB
Família de colunas	Coluna versionada	Escrita massiva distribuída com leitura por chave	Cassandra, HBase, Bigtable
Grafo	Nó e aresta	Percursos e conexões de profundidade variável	Neo4j, Neptune
Série temporal	Ponto (tempo, valor)	Métricas e sensores ao longo do tempo	InfluxDB, TimescaleDB
Vetorial	Vetor de embeddings	Busca por similaridade semântica (IA generativa, RAG)	pgvector, Qdrant, Milvus



Muitos sistemas reais combinam vários: relacional para o núcleo transacional, chave-valor para cache, colunar para análise, vetorial para busca semântica. Isso se chama persistência poliglota.

Dados estruturados vs. não estruturados

	Estruturados	Não estruturados
Esquema	Claramente definido	Qualquer coisa
Análise	Fáceis de buscar e analisar	Chegam rápido — não é preciso limpar antes de guardar
Abrangência	Só aceitam dados que caibam no esquema	Aceitam dados de qualquer origem
Mudança de esquema	Causa muita dor de cabeça	Não há esquema para mudar
Onde vivem	Data warehouse (armazém de dados)	Data lake (lago de dados)



A frase que resume tudo: no dado estruturado a estrutura é assumida na ESCRITA (schema on write); no não estruturado, na LEITURA (schema on read).

Quem paga a conta da bagunça, e quando

Schema on write — o armazém

A estrutura é exigida na entrada. Se o dado não se encaixa, ele é rejeitado.

Quem paga: quem produz o dado, na hora de escrever.

Resultado: entrada mais lenta e trabalhosa, leitura confiável e rápida para todo mundo depois.

Schema on read — o lago

Guarda-se tudo como veio. A estrutura só é aplicada quando alguém lê.

Quem paga: quem consome o dado, toda vez que consome.

Resultado: entrada instantânea; risco de virar um pântano de dados que ninguém sabe interpretar.

A síntese atual: lakehouse

Guardar em formato aberto e colunar (Parquet) no armazenamento barato do lago, e colocar por cima uma camada de metadados e transações (Delta Lake, Apache Iceberg, Hudi) que devolve esquema, versionamento e garantias transacionais. É o desenho dominante nas arquiteturas de dados atuais — e o tema do módulo de armazenamento de BD2.



Pergunta de projeto: o seu dado será lido por cinco pessoas ou por cinquenta times? Quanto mais consumidores, mais vale pagar a conta uma vez na escrita.

Que modelo você usaria?

a

a) Relacional

Tabelas, chaves, junções, transações.

b

b) Documento

Registros JSON autocontidos.

c

c) Grafo

Nós e arestas tipadas.

d

d) Chave-valor

Busca direta por identificador.

?

Cenário: um portal de transparência precisa responder "quais empresas estão ligadas, direta ou indiretamente, a esta pessoa por meio de sociedade?" — sabendo que a cadeia societária pode ter 2 ou 9 níveis, e ninguém sabe de antemão.

O mesmo domínio, três modelos

Domínio sugerido: um sistema de matrícula (aluno, disciplina, professor, turma, pré-requisito).

Passo	O que fazer
1	Modele o domínio em relacional: desenhe as tabelas normalizadas e as chaves
2	Modele o mesmo domínio em documento: decida o que fica aninhado e o que fica separado
3	Modele em grafo: escolha o que vira nó e o que vira aresta
4	Escreva 3 consultas em português: uma simples, uma agregada e uma sobre cadeia de pré-requisitos
5	Diga, para cada consulta, em qual modelo ela é mais fácil — e por quê
6	Conclusão: qual modelo você escolheria e qual dor você aceita ao escolher



O passo 6 é o mais importante: toda escolha de modelo é uma dor aceita conscientemente. Nomear a dor é o que separa a decisão técnica do chute.

BLOCO 4

Engines de armazenamento e processamento

- - Processamento transacional (OLTP)
 - Garantias ACID
 - Processamento analítico (OLAP)
 - Desacoplamento entre armazenar e processar

Duas cargas de trabalho, dois mundos

Até aqui falamos de representação. Agora falamos de implementação: como o banco de fato grava, indexa e lê os bytes — e para que tipo de carga ele foi otimizado.

1

Carga transacional

Muitas operações pequenas, feitas o tempo todo, quase sempre sobre um registro por vez.

"Registre esta compra." "Atualize este cadastro." "Crie esta corrida."

Banco otimizado para isso: OLTP.

2

Carga analítica

Poucas operações enormes, feitas de vez em quando, varrendo milhões de registros.

"Qual o ticket médio do último trimestre por região?"

Banco otimizado para isso: OLAP.

!

Colocar carga analítica pesada no banco transacional de produção é um erro clássico: a consulta gigante de relatório derruba o desempenho do sistema que atende usuários.

OLTP — processamento transacional em linha

O que é uma transação aqui

Qualquer ação registrada pelo sistema: publicar um post, pedir uma corrida, subir uma nova versão de modelo, emitir uma nota. Não precisa envolver dinheiro.

Operações típicas

Inserir no momento em que o fato acontece; ocasionalmente atualizar ou apagar. Muitas escritas pequenas, muitas leituras pontuais por chave.

Requisito: baixa latência

A pessoa está esperando na tela. Milissegundos importam; segundos são inaceitáveis.

Requisito: alta disponibilidade

Se o banco cai, o produto cai junto. Ninguém consegue pedir a corrida, publicar o post nem emitir a nota.



Como cada operação mexe em um registro inteiro, o armazenamento orientado a LINHA cai como uma luva — e é o padrão nesses bancos.

ACID: as quatro garantias clássicas

A — Atomicidade

Todos os passos de uma transação falham ou têm sucesso como um bloco.

Exemplo: se o pagamento falhar, não designe motorista. Nunca meio caminho.

C — Consistência

A transação leva o banco de um estado válido a outro estado válido, respeitando as regras declaradas.

Exemplo: o saldo nunca fica negativo se a regra proíbe.

I — Isolamento

Transações simultâneas produzem o mesmo resultado que produziram em sequência.

Exemplo: não designe o mesmo motorista para dois pedidos feitos ao mesmo tempo.

D — Durabilidade

Uma vez confirmada, a transação sobrevive a queda de energia, reinício e falha de máquina.

Exemplo: "pedido confirmado" continua confirmado depois do reboot.



Nuance que o deck original destaca: ACID não é obrigatório. Vários sistemas transacionais distribuídos relaxam alguma dessas garantias em troca de escala e disponibilidade (modelo BASE) — e aí a responsabilidade por evitar o motorista duplicado sobe para a sua aplicação. A pergunta certa não é "tem ACID?", e sim "de que garantia eu posso abrir mão, e quem cobre o buraco?".

Por que OLTP é tipicamente orientado a linha

Olhe a operação mais comum de um sistema transacional: ela toca todos os atributos de um único registro.

```
INSERT INTO Corrida (CorridaID, Usuario, MotoristaID, Cidade, Mes, Preco)
VALUES ('10', 'memelord', '3932839', 'Brasília', 'julho', 20.40);
```

CorridaID	Usuario	MotoristaID	Cidade	Mes	Preco
10	memelord	3932839	Brasília	julho	20,40
...	...	...	...	...	...

A escrita toca uma LINHA inteira — guardá-la contígua em disco torna a operação uma só gravação.



Se esse mesmo registro estivesse guardado por coluna, uma única inserção precisaria escrever em seis lugares diferentes do disco.

OLAP — processamento analítico em linha

A pergunta que ele responde

Como obter informação agregada a partir de um volume grande de dados? Exemplo: qual foi o preço médio das corridas do mês passado para quem embarcou na UnB?

Operações típicas

Predominantemente SELECT, com agregações (SUM, AVG, COUNT), agrupamentos e janelas. Escrita quase sempre em lote, não registro a registro.

Requisito: consultas complexas em muito dado

Precisa varrer milhões ou bilhões de registros e ainda assim entregar um número correto.

Requisito: tempo de resposta tolerável

Segundos, minutos, às vezes horas. Ninguém está olhando a tela travada esperando o relatório trimestral.



Repare no contraste com o OLTP: lá, muitas operações minúsculas com prazo de milissegundos; aqui, poucas operações gigantescas com prazo de minutos.

Por que OLAP é tipicamente orientado a coluna

Agora olhe a operação mais comum de um sistema analítico: ela toca poucos atributos de milhões de registros.

```
SELECT AVG(Preco)
FROM Corrida
WHERE Cidade = 'Brasília' AND Mes = 'julho';
```

CorridaID	Usuario	MotoristaID	Cidade	Mes	Preco
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...

A consulta toca 3 de 6 COLUNAS — guardadas por coluna, as outras três nem chegam a ser lidas do disco.



Em uma tabela com 200 colunas, ler 3 em vez de 200 não é uma otimização marginal: é a diferença entre a consulta rodar em segundos ou em horas.

OLTP vs. OLAP em uma tabela

Critério	OLTP (transacional)	OLAP (analítico)
Pergunta típica	"Registre esta corrida."	"Qual o preço médio das corridas em julho?"
Operação predominante	INSERT, UPDATE, DELETE pontuais	SELECT com agregação
Volume por operação	Um registro	Milhões de registros
Latência aceitável	Milissegundos	Segundos a horas
Concorrência	Milhares de operações simultâneas	Poucas consultas simultâneas, cada uma pesada
Orientação de armazenamento	Linha	Coluna
Modelagem usual	Normalizada	Desnormalizada (estrela, floco de neve)
Exemplos	PostgreSQL, MySQL, SQL Server	BigQuery, Snowflake, Redshift, DuckDB, ClickHouse



Cuidado com o nome: "OnLine" nos dois casos significa apenas "interativo, não em fita", herança dos anos 1990. Não tem relação com internet.

Por que OLTP e OLAP são termos envelhecidos

Os conceitos continuam úteis; os rótulos, nem tanto. Quatro razões:

1. "OnLine" perdeu o sentido

Nasceu para distinguir processamento interativo de processamento em fita magnética. Hoje tudo é interativo — o adjetivo não separa mais nada.

2. A separação virou arquitetura, não produto

Bancos transacionais modernos rodam análises razoáveis; motores analíticos aceitam atualizações. A fronteira ficou porosa (HTAP).

3. Os termos misturam armazenamento e processamento

Dizer "é um banco OLAP" descreve ao mesmo tempo como o dado está guardado e como ele é processado — e essas duas coisas se separaram.

4. Falta o meio de campo

Nenhum dos dois nomes cobre streaming, aprendizado de máquina, busca vetorial ou processamento de grafos — cargas hoje corriqueiras.



Use os termos como atalho para descrever a CARGA de trabalho ("essa consulta é analítica"), não como rótulo definitivo de um produto.

Desacoplar armazenamento de processamento

ANTES — cada carga, seu próprio banco

Banco OLTP

armazena por linha
e processa por linha

Banco OLAP

armazena por coluna
e processa por coluna

O mesmo dado é copiado
para vários bancos, cada um
com seu motor acoplado.

DEPOIS — um armazenamento, vários motores

Camada de armazenamento — barata, elástica, em formato aberto (Parquet em object storage)

Motor SQL

Motor de ML

Motor de streaming

Motor de busca

Escala armazenamento e computação
separadamente; paga-se só o que roda.

[Snowflake](#), [BigQuery](#), [lakehouse](#)



Por que isso importa para a sua decisão: quando armazenar é barato e processar é caro, a pergunta deixa de ser "qual banco comprar" e passa a ser "que motor eu aponto para os meus dados, e quando".

Onde essa consulta deve rodar?

a

a) No banco de produção

Direto na base transacional que atende o aplicativo.

b

b) Em um armazém colunar

Réplica analítica alimentada periodicamente.

c

c) Em uma planilha

Exportar tudo e agregar à mão.

d

d) Em cache chave-valor

Guardar o resultado pronto por chave.

?

Cenário: a diretoria quer um painel que cruza 18 meses de pedidos com cadastro de clientes e histórico de atendimento, atualizado uma vez por dia. São 400 milhões de linhas e 12 agregações.

BLOCO 5

Movimentação de dados



- ETL: extrair, transformar, carregar
- O que exatamente é "transformar"
- Processamento em lote e contínuo
- O pêndulo ETL → ELT → ETL

ETL: extrair, transformar, carregar

O caminho clássico entre o mundo transacional e o mundo analítico.



Extrair e carregar são mecânicos. O trabalho de verdade — e a maior parte do código e dos erros — está no T.



Historicamente esse processo roda em lote (batch): uma vez por hora, uma vez por noite. Daí a expressão "os dados de ontem" nos relatórios.

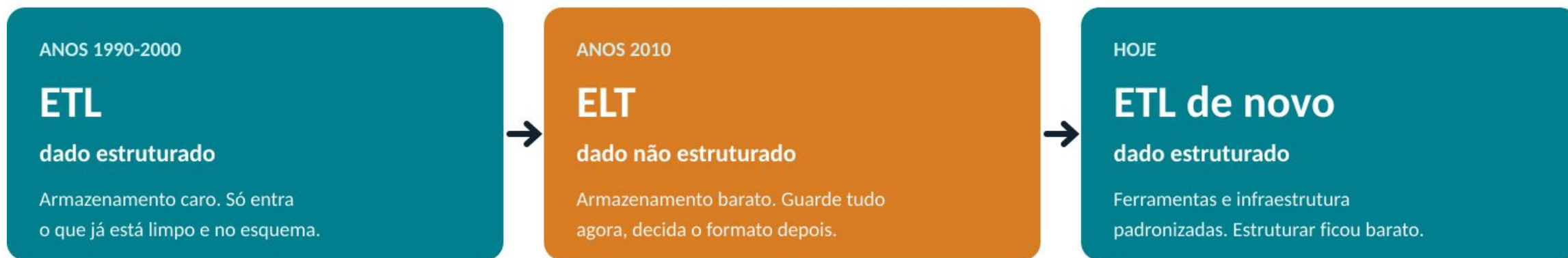
Transformar: a parte substantiva

Operação	O que faz	Exemplo concreto
Limpar	Corrige e descarta o que está errado	Data "31/02", CPF com letra, preço negativo
Validar	Rejeita o que não obedece às regras	Município fora da lista oficial do IBGE
Transpor / remodelar	Muda a forma da tabela	De uma coluna por mês para uma linha por mês
Derivar valores	Cria campos calculados	Idade a partir da data de nascimento; faixa etária
Juntar fontes	Combina dados de origens diferentes	Pedido do sistema interno + cadastro de um serviço externo
Deduplicar	Elimina registros repetidos	O mesmo evento entregue duas vezes pela fila
Dividir	Separa um campo em vários	"Brasília/DF" vira cidade e unidade federativa
Agregar	Resume em um nível mais alto	Vendas por dia a partir de vendas por pedido



Regra de ouro: cada transformação é uma decisão de negócio disfarçada de código. "Descartei os pedidos sem CEP" muda o número que a diretoria vai ler. Documente.

O pêndulo: ETL → ELT → ETL



A força que empurrou para o ELT: queríamos mais flexibilidade.

A força que trouxe de volta ao ETL: as ferramentas e a infraestrutura se padronizaram — estruturar deixou de ser caro.



A diferença entre os dois é só a ordem das letras — mas ela decide onde a complexidade mora: no pipeline de entrada (ETL) ou na hora de consultar (ELT).

ETL ou ELT? E em lote ou contínuo?

Escolha ETL quando...	Escolha ELT quando...
O destino é caro ou tem esquema rígido	O armazenamento é barato e aberto (lago / lakehouse)
Há dado pessoal a anonimizar antes de carregar	Ainda não se sabe todas as perguntas que serão feitas
As regras de negócio são estáveis e conhecidas	As regras mudam com frequência e é preciso reprocessar
O volume é moderado e a qualidade é crítica	O volume é alto e a velocidade de ingestão é crítica

Em lote (batch)

O pipeline roda de tempos em tempos: de hora em hora, toda madrugada. Simples de operar, fácil de reprocessar, barato. É o padrão — e resolve a maioria dos casos.

Contínuo (streaming)

O dado é processado à medida que chega. Necessário para detecção de fraude, alertas e recomendação ao vivo. Custa muito mais em complexidade operacional.



Comece em lote. Migre para contínuo só quando alguém conseguir dizer quanto custa o atraso de uma hora.

Desenhe o pipeline de ponta a ponta

Retome o sistema que você inventariou no exercício do bloco 1 e feche o ciclo.

Passo	O que entregar
1	Um diagrama: fontes → ingestão → armazenamento → transformação → consumo
2	Para cada caixa, o formato do dado (CSV, JSON, Parquet...) e por quê
3	Para cada armazenamento, o modelo (relacional, documento, grafo...) e a carga (transacional ou analítica)
4	A escolha entre ETL e ELT, com o critério que decidiu
5	A escolha entre lote e contínuo, com o custo do atraso declarado
6	Um risco: qual peça, se falhar, derruba o resto — e o que você faria a respeito



Esse diagrama, com as justificativas dos passos 2 a 5, é praticamente um ADR pronto. É assim que a disciplina avalia decisão técnica: pela justificativa, não pela ferramenta escolhida.

A aula inteira em uma página

1 • FONTES

- Usuário: mal formatado, urgente
- Sistema: padronizado, volumoso
- Interno: o que a casa já tem
- Terceiro: útil, arriscado, regulado

2 • FORMATOS

- Serializar para poder reconstruir
- Texto: legível e volumoso
- Binário: compacto e opaco
- Linha = amostra · Coluna = atributo

3 • MODELOS

- Relacional: normalizar evita anomalia
- SQL é declarativo; o motor decide o como
- Documento: autocontido e flexível
- Grafo: a relação é o dado

4 • ENGINES

- OLTP: muitas operações minúsculas
- ACID: garantia, não obrigação
- OLAP: poucas operações enormes
- Armazenar e processar se separaram

5 • MOVIMENTAÇÃO

- E e L são mecânicos; o T é o trabalho
- Transformar é decisão de negócio
- ETL → ELT → ETL: um pêndulo
- Comece em lote; streaming custa caro

O FIO CONDUTOR

Não existe melhor banco, melhor formato nem melhor modelo.

Existe a escolha coerente com a fonte, o padrão de acesso e a carga de trabalho — e justificada.

Você deve conseguir explicar, em voz alta:

- A diferença entre dado gerado por usuário e dado gerado por sistema — e por que a urgência de processamento muda
- O que é serialização, e por que formato de dados é "um padrão acordado"
- Quando um formato orientado a coluna vence um orientado a linha — e por quê
- Por que 1.000.000 ocupa mais espaço em CSV do que em Parquet
- O que a normalização resolve, e o preço que ela cobra
- Por que SQL ser declarativo permite que o banco fique mais rápido sem você reescrever a consulta
- Quando o modelo de grafo torna trivial uma consulta que o relacional torna sofrida
- A diferença entre schema on write e schema on read, e quem paga a conta em cada um
- As quatro letras de ACID, com um exemplo de cada
- Por que um relatório pesado não deve rodar no banco de produção
- O que significa desacoplar armazenamento de processamento, e o que isso muda na conta do fim do mês
- A diferença entre ETL e ELT, e um critério objetivo para escolher entre os dois

Glossário: você vai ler tudo isso em inglês

Português	Inglês	Português	Inglês
Fonte de dados	data source	Armazém de dados	data warehouse
Serialização	serialization	Lago de dados	data lake
Orientado a linha	row-major	Esquema na escrita	schema on write
Orientado a coluna	column-major	Esquema na leitura	schema on read
Tupla / linha	tuple / row	Processamento em lote	batch processing
Atributo / coluna	attribute / column	Processamento contínuo	stream processing
Normalização	normalization	Extrair, transformar, carregar	extract, transform, load
Junção	join	Otimização de consultas	query optimization
Chave estrangeira	foreign key	Persistência poliglota	polyglot persistence



Dica: aprenda os termos nos dois idiomas. A documentação, os erros e as vagas de emprego estão em inglês; a discussão em sala e o relatório, em português.

Para ir além

Origem deste material

HUYEN, Chip. "Data Engineering 101" — CS 329S: Machine Learning Systems Design, Stanford University.

HUYEN, Chip. Designing Machine Learning Systems. O'Reilly, 2022 — o capítulo 3 desenvolve tudo o que vimos hoje.

Bibliografia da disciplina

REIS, Joe; HOUSLEY, Matt. Fundamentos de Engenharia de Dados. Novatec, 2023 — referência principal do curso.

KLEPPMANN, Martin. Designing Data-Intensive Applications. O'Reilly — o capítulo 2 aprofunda modelos de dados; o 3, engines de armazenamento.

Complementos

- KIMBALL, Ralph; ROSS, Margy. The Data Warehouse Toolkit — modelagem dimensional (estrela, floco de neve).
- ARMBRUST, Michael et al. "Lakehouse: A New Generation of Open Platforms..." CIDR, 2021 — a síntese lago + armazém.
- SILBERSCHATZ; KORTH; SUDARSHAN. Sistema de Banco de Dados — fundamentos do modelo relacional.
- chiphuyen/just-pandas-things — por que iterar linhas em DataFrame é lento.
- Documentação oficial do Apache Parquet, do PostgreSQL e do MongoDB.

A pergunta que fica

"Qual banco eu uso?" é a pergunta errada.

A pergunta certa é: de onde vem meu dado, como ele vai ser lido, com que garantia e a que custo — e o que eu registro como justificativa da escolha.

Próximo módulo: método de decisão e ADR

Traga o inventário do bloco 1

Leitura: Reis & Housley, cap. 1-3